

AgriVision AI

Project Report

System workflow, AI diagnosis, technology stack, implementation statistics,
and deployment architecture

Implementation review of the uploaded AgriVision-AI workspace

Contents

1	Executive summary.....	1
2	Product scope and user workflow.....	1
3	Implemented functions.....	2
3.1	Dashboard and navigation.....	2
3.2	Leaf scan interface.....	2
3.3	AI diagnosis endpoint.....	3
3.4	Accessibility and resilience.....	3
4	Technology stack.....	3
5	Project statistics.....	4
6	AI behavior, safety, and limitations.....	6
7	Deployment workflow.....	6
8	Verification status.....	7
9	Conclusion.....	7
10	References.....	8

1 Executive summary

AgriVision AI is a farmer-oriented web application for scanning crop leaves and returning an uncertainty-aware health assessment. The interface combines a warm agricultural visual system with an animated 3D scan surface. The implemented scan workflow accepts a camera or gallery image, compresses it in the browser, derives basic image-quality signals, requests a server-side vision analysis, and renders the returned diagnosis dynamically.

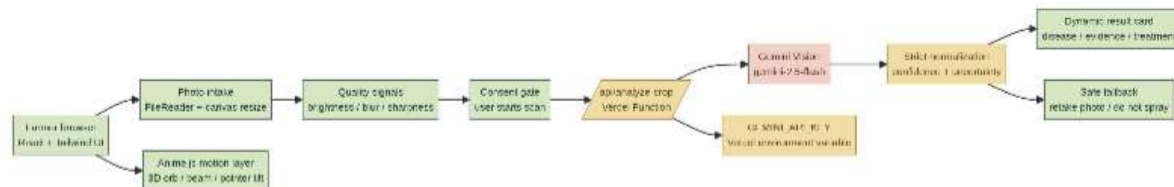
The most important product change in the reviewed implementation is the replacement of a simulated scan timer and hard-coded wheat-rust result with a real AI request path. The browser now posts the consented image to `/api/analyze-crop`. A Vercel serverless function keeps `GEMINI_API_KEY` private, sends the image to Gemini Vision, normalizes the response, and returns either a structured diagnosis or a safe uncertain result. The UI displays the model confidence, disease names, estimated yield impact, visible evidence, and treatment guidance rather than presenting a fixed example.

The project is technically positioned as a Vite-powered React single-page application with TypeScript, Tailwind CSS, Anime.js motion, Lucide icons, and a Vercel serverless API. The original workspace also contains an Express API artifact and generated OpenAPI/Zod clients. The Vercel deployment path uses the dedicated `api/analyze-crop.ts` function and the `vercel.json` build and routing configuration.

Area	Finding	Interpretation
Product behavior	Implemented	A real uploaded image is sent to the AI endpoint and the result card is data-driven.
AI safety	Implemented	Low confidence produces an uncertain response and photo-taking guidance.
Deployment	Implemented	Vercel configuration includes a Vite build, static output, serverless function, and SPA rewrite.
Preview status	Environment-dependent	The sandbox preview is reachable, but Gemini requires <code>GEMINI_API_KEY</code> in its runtime.

2 Product scope and user workflow

The product is designed around a short field decision loop. A farmer opens the dashboard, navigates to the crop scan section, captures or selects a leaf image, confirms consent, and starts the scan. The interface provides visual progress through the animated orb and scanning beam. When the server responds, the right-hand result card changes from trust and privacy messaging to the actual assessment.



1 AgriVision AI end-to-end scan and deployment architecture.

The workflow is intentionally divided into client-side preparation and server-side inference. The browser reads the image with `FileReader`, draws it to a canvas, limits its maximum dimension to 1,280 pixels,

calculates brightness and edge-based sharpness signals, and compresses the image as JPEG. The request includes the compressed base64 image, MIME type, selected language, and quality signals. The server does not trust the filename or the client-side quality values as a diagnosis; they are contextual signals for the prompt.

Step	Component	Function
1	Photo intake	Camera or gallery image is selected through an image input with environment capture support.
2	Canvas preprocessing	Image is resized, converted to JPEG, and transformed into a request-safe base64 payload.
3	Consent gate	The scan cannot start until the user explicitly agrees to the image-processing flow.
4	AI request	The browser calls <code>/api/analyze-crop</code> ; the secret key is not exposed to client code.
5	Diagnosis normalization	The server clamps confidence and yield values, validates severity, and supplies safe defaults.
6	Result rendering	The result card renders disease, confidence, impact, reasoning, and treatment steps.
7	Fallback	Unavailable or weak evidence becomes an uncertain result with retake guidance.

3 Implemented functions

3.1 Dashboard and navigation

The main application provides a responsive public-facing dashboard with sections for today's fields, crop scanning, fields, scan history, treatment guidance, and yield estimation. It supports English and Hindi copy, with the underlying language model also accepting Braj-oriented instructions. The visual system uses a dark crop-green primary, warm ochre highlights, coral warning accents, serif display typography, and a monospaced metadata style.

3.2 Leaf scan interface

The scan interface is the central functional feature. It presents an animated circular scanner with nested rings, a leaf or uploaded image inside the orb, a progress beam, and status text that changes between ready, scanning, result, and error states. Anime.js controls the entrance animation, scan rotation, and pointer-based tilt. CSS handles the persistent scan beam, floating leaf treatment, pulse dots, and reduced-motion behavior.

The interface also includes visible privacy language, an explicit local-consent checkbox, camera/gallery upload controls, a scan button, an error state, and a rescan state. The result card no longer assumes a particular crop or disease. It uses the response fields and shows a confidence percentage, uncertainty mode, yield impact, visible evidence, and treatment list.

3.3 AI diagnosis endpoint

The serverless function validates the request shape, limits the image payload, reads `GEMINI_API_KEY`, and sends the image as inline data to the Gemini `gemini-2.5-flash` model. The prompt instructs the model to return one JSON object with a fixed set of fields. It also instructs the model not to invent pesticide dosages and not to claim laboratory certainty.

The normalization layer is important because a vision model can produce incomplete or poorly typed output. Confidence is clamped to the interval 0–1. Yield ranges are clamped to 0–100 percent. Severity is restricted to `healthy`, `low`, `moderate`, `high`, or `unknown`. Empty disease values become null. Missing treatment and explanation fields receive conservative fallback text. A result is labelled reliable only when a disease is present and confidence is at least 0.6.

3.4 Accessibility and resilience

The interface includes keyboard-visible focus styles, responsive layouts, semantic buttons, alternative text for the uploaded image, reduced-motion handling, and a readable result hierarchy. The server returns explicit error messages for invalid image payloads, oversized images, missing configuration, provider errors, and temporary unavailability.

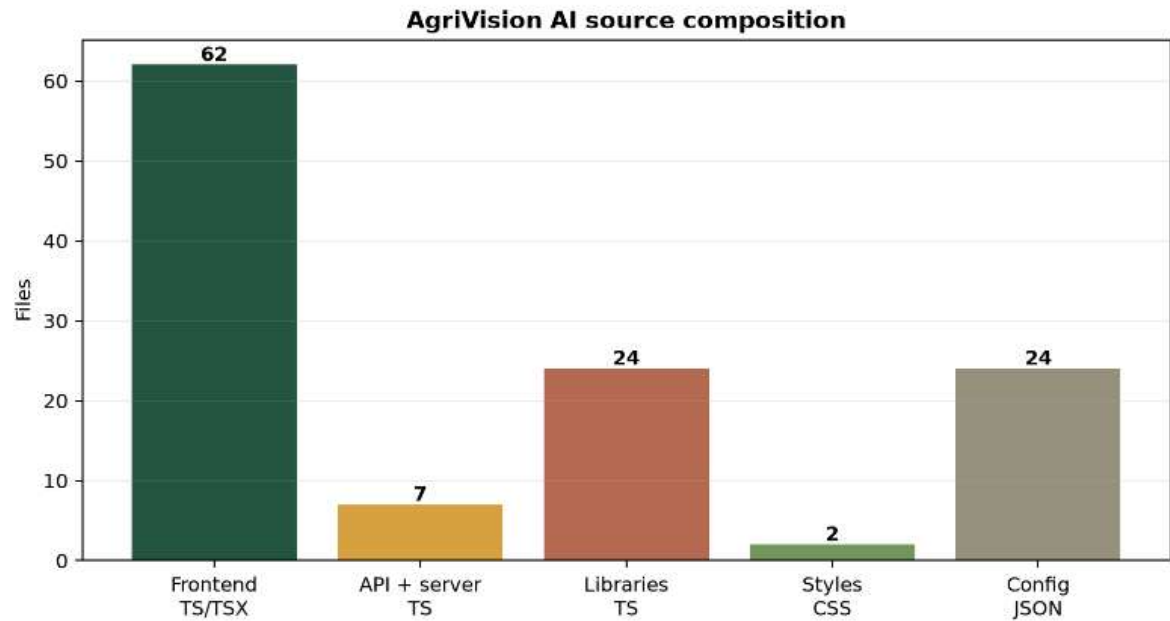
4 Technology stack

Layer	Technology	Role in the project
UI	React + TypeScript	Component-based single-page application and typed interaction state.
Build	Vite	Fast local development and production bundling for the frontend.
Styling	Tailwind CSS 4	Responsive utility styling and design-token-driven visual language.
Motion	Anime.js	3D-feeling orb rotation, entrance, pointer tilt, and scan-state motion.
Icons	Lucide React	Consistent interface icons for scanning, privacy, history, guidance, and actions.
AI	Google Gemini Vision	Multimodal leaf-image analysis through <code>gemini-2.5-flash</code> .
API	Vercel Functions	Server-side secret handling and serverless request execution.
Contracts	OpenAPI + Zod generated clients	Typed crop-analysis input and output definitions in the workspace.
Package management	pnpm workspace	Monorepo dependency and package orchestration.

The application also contains an Express API artifact under `artifacts/api-server` and generated packages under `lib/api-client-react`, `lib/api-spec`, and `lib/api-zod`. The Vercel route is deliberately self-contained so the deployment can run the image-analysis function without exposing the API key in the React bundle.

5 Project statistics

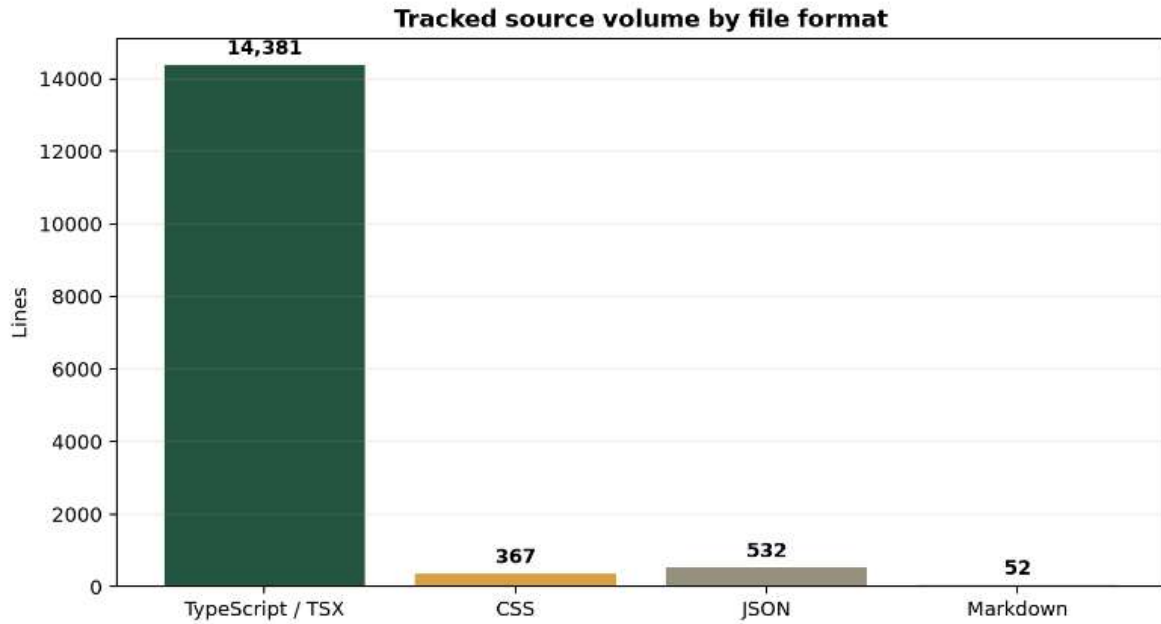
The following statistics are measured from the uploaded workspace while excluding Git metadata, installed dependencies, and generated production output. They describe implementation size, not user adoption or model accuracy.



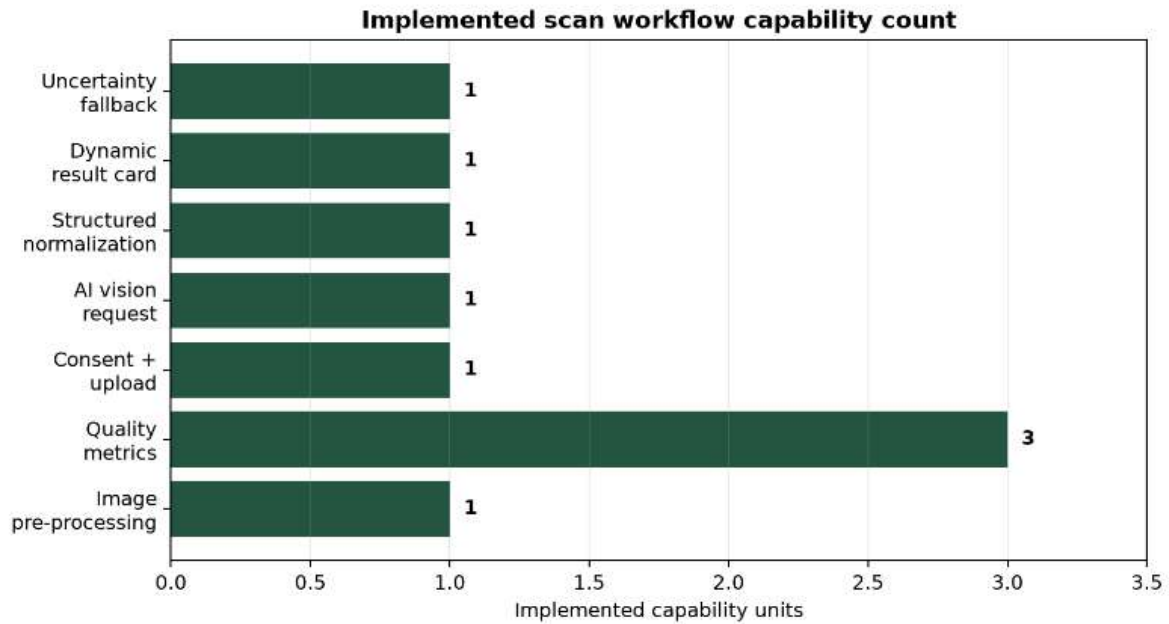
2 Source-file composition measured from the workspace.

Metric	Value	Definition
TypeScript files	156	All tracked <code>.ts</code> and <code>.tsx</code> files in the workspace.
TypeScript lines	14,381	Non-empty and empty source lines counted from tracked TypeScript files.
Frontend TS/TSX files	62	Files under the primary AgriVision frontend source tree.
API/server TS files	7	Files in the Vercel API and Express API artifact areas.
Library TS files	24	Files under the workspace library packages.
CSS files / lines	2 / 367	Global and component styling sources.

JSON files	24	Workspace configuration and generated contract metadata.
Anime.js calls	6	Direct Anime.js invocation sites in the main UI.
API routes detected	2	Health and crop-analysis routes in the Express artifact.



3 Tracked source volume by file format.



4 Implemented scan workflow capability units.

The capability chart counts implemented feature units in the scan path. It is not a performance benchmark. The project does not currently contain a production telemetry dataset, labelled leaf-image test set, measured accuracy report, or user-volume history. Those metrics should be added before making claims about diagnostic accuracy or operational scale.

6 AI behavior, safety, and limitations

AgriVision AI is a decision-support interface, not a laboratory diagnostic instrument. The system can identify visual patterns that resemble crop disease, nutrient stress, pest damage, or healthy foliage, but image quality, crop variety, lighting, disease co-occurrence, and regional differences can materially affect the result.

The implementation addresses the most important product risk by making uncertainty explicit. If the model returns weak evidence or the response cannot be normalized, the UI presents an uncertain state and tells the user to retake the image. The endpoint does not provide invented chemical dosage when the model does not supply a treatment list. The report should not be interpreted as evidence that a particular disease classification is correct for every field condition.

The current environment also has a deployment distinction. The Vercel build is designed to call Gemini when `GEMINI_API_KEY` is present in Vercel project settings. The temporary sandbox preview can serve the UI, but it does not automatically inherit the user's Vercel environment variables. Therefore, the preview is suitable for interface review, while production AI behavior should be validated after configuring the key in Vercel and redeploying.

7 Deployment workflow

The intended deployment sequence is straightforward. The repository is imported into Vercel. Vercel installs the pnpm workspace, runs the frontend build command, publishes `artifacts/agrivision-ai/dist/public`, deploys `api/analyze-crop.ts` as a serverless function, and applies the SPA rewrite for non-API routes. The only required secret is `GEMINI_API_KEY`, configured in Vercel under project environment variables.

The deployment file sets a 60-second function duration and routes all non-API paths to `index.html`. The Vite configuration now defaults `PORT` to 5173 and `BASE_PATH` to `/`, which removes the prior dependency on Replit-only environment variables during a Vercel build. The API function does not persist uploaded images; the request is processed in memory and the response is returned to the browser.

Deployment item	Configuration	Operational meaning
Install	<code>pnpm install --frozen-lockfile</code>	Reproducible workspace dependency installation.
Build	<code>pnpm --filter workspace/agrivision-ai build</code>	Builds the Vite frontend into the configured public directory.
Static output	<code>artifacts/agrivision-ai/dist/public</code>	Directory served by Vercel as the website.
Function	<code>api/analyze-crop.ts</code>	Secure server-side Gemini request path.

Secret	GEMINI_API_KEY	Must be added in Vercel environment variables.
Rewrite	All non-API paths to / index.html	Preserves client-side navigation on refresh.

8 Verification status

The reviewed implementation passed the frontend TypeScript check and production Vite build. The standalone Vercel function also passed TypeScript validation after adding declarations for the Node/Vercel runtime APIs used by the function. The build produced the expected HTML, JavaScript, CSS, favicon, and robots files.

The build emitted a non-blocking source-map warning from the existing tooltip component. It did not fail the build or prevent the production bundle from being generated. A final production verification should include a real leaf image, a configured Vercel key, a successful Gemini response, an uncertain or low-quality image, and a provider-error path.

9 Conclusion

AgriVision AI has a coherent product shell and a complete first-pass technical path for AI-assisted leaf assessment. Its strongest implemented qualities are the short farmer-centered workflow, the responsive and animated visual system, server-side secret handling, typed result normalization, and uncertainty-aware presentation. Its next validation milestone is not additional visual decoration; it is a small labelled image evaluation set and production telemetry that can measure confidence calibration, response latency, provider failure rate, and agreement with local agricultural experts.

10 References

The implementation and report are grounded in the uploaded source workspace. The external references below document the principal technologies used by the system.

- [1] Google AI for Developers, “Image understanding | Gemini API.” <https://ai.google.dev/gemini-api/docs/image-understanding>
- [2] Vercel, “Vercel Functions.” <https://vercel.com/docs/functions>
- [3] Vercel, “Project Configuration.” <https://vercel.com/docs/project-configuration>
- [4] Anime.js, “Documentation | JavaScript Animation Engine.” <https://animejs.com/documentation>
- [5] Anime.js, “Anime.js | JavaScript Animation Engine.” <https://animejs.com/>